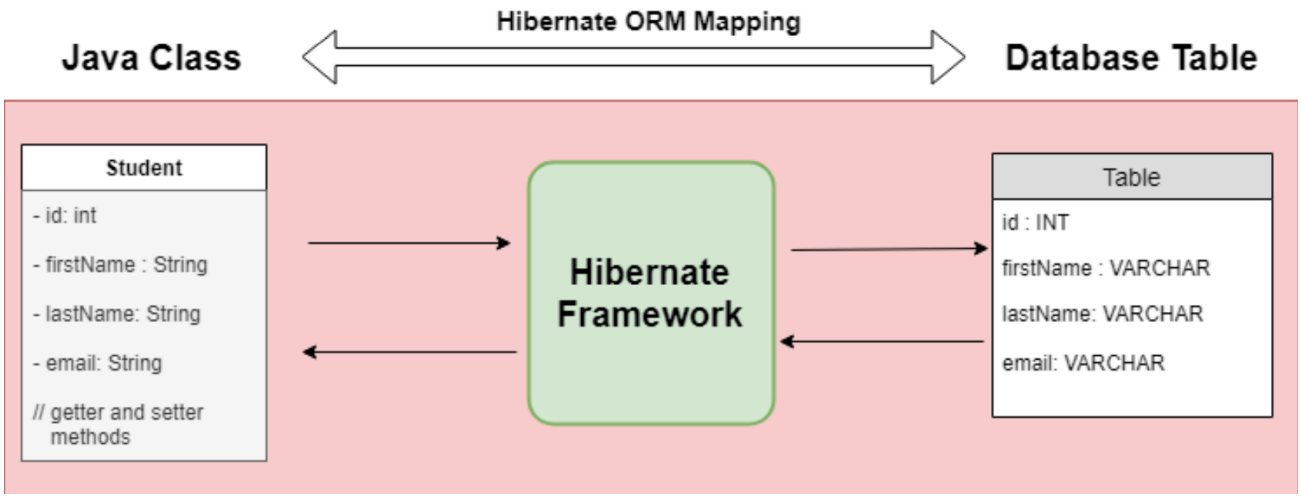
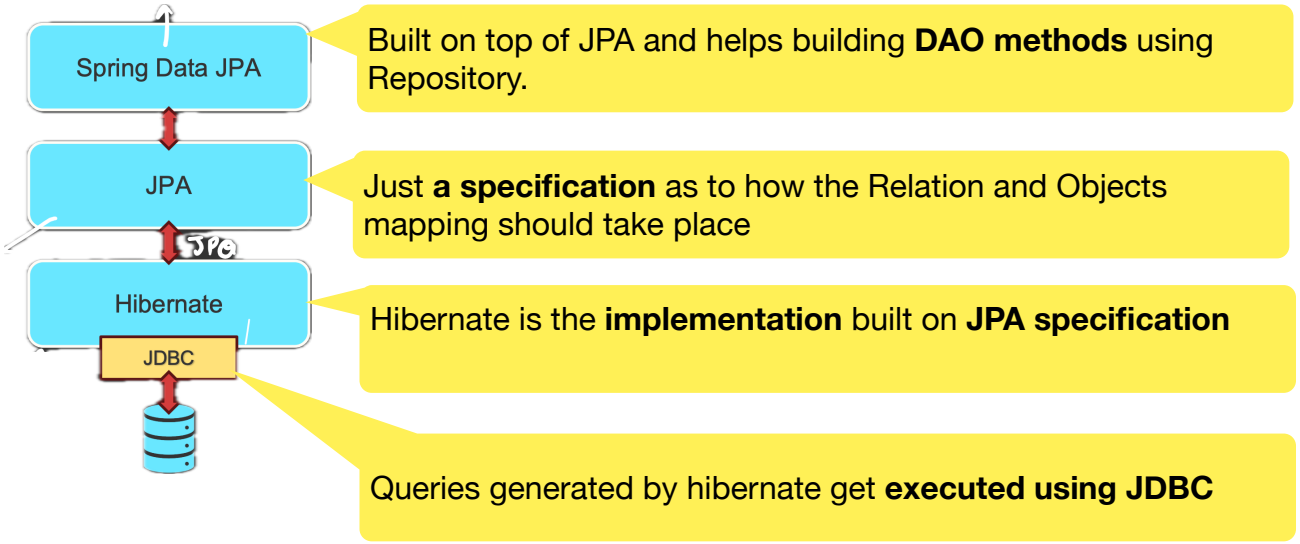
3.2 Hibernate ORM, JPA, Entities and Tables in Spring Data JPA

- **Hibernate ORM Mapping:**





- **Hibernate:**
  - Hibernate is a powerful, high-performance **Object-Relational Mapping** (ORM) **framework** that is widely used with Java.
  - It provides a framework for mapping an object-oriented domain model to a **relational database**.
  - Hibernate is one of the **implementations** of the **Java Persistence API** (JPA), which is a standard specification for **ORM** in Java.

3.2 Hibernate ORM, JPA, Entities and Tables in Spring Data JPA

- **JPA (Java Persistence API):**
  - **JPA** is a **specification** for object-relational mapping (ORM) in Java.
  - It defines a set of **interfaces** and **annotations** for mapping **Java objects** to **database tables** and vice versa.
  - JPA itself is just **a set of guidelines** and does not provide any implementation.
  - The implementation of JPA is provided by **ORM frameworks** such as **Hibernate**, **EclipseLink**, and **OpenJPA**.

JPA Provides a **standard** for **ORM** in Java applications, ensuring that developers can switch between different **JPA providers** without changing their code. And **Hibernate** is one such JPA Provider.

However, **Hibernate** is a specific **implementation** of **JPA** and a powerful **ORM framework** on its own. It offers additional features and optimizations beyond the JPA specification, making it a popular choice for **ORM** in Java applications.

- **Common Hibernate Configurations:**
  1. spring.jpa.hibernate.ddl-auto=update/create/validate/createdrop/none
     - Controls how **Hibernate** handles database schema generation.
       - **none** ⇒ Do Nothing.
       - **create** ⇒ Drop tables & create again (data lost).
       - **create-drop** ⇒ Create on start, drop on shutdown of server.
       - **update** ⇒ Update schema safely (most common in dev).
       - **validate** ⇒ Only validate schema, no changes.
  2. spring.jpa.show-sql=true
     - Prints **SQL queries generated** by **Hibernate** in the **console**.
     - Very helpful for **learning JPA/Hibernate**.
  3. spring.jpa.properties.hibernate.format_sql=true
     - Formats SQL queries in a **readable way**.
  4. spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5Dialect (Optional)
     - Different **databases** have different **SQL syntax**:
       - MySQL → LIMIT
       - Oracle → ROWNUM
       - PostgreSQL → LIMIT OFFSET
     - Tells Hibernate **which database** it is **talking** to and how to:
       - Generate **correct SQL**
       - Handle **pagination**
       - Choose **correct data types**

3.2 Hibernate ORM, JPA, Entities and Tables in Spring Data JPA

- **JPA / Hibernate Entity Annotations:**

Entity annotations are used to map **Java classes and fields** to **database tables and columns**. Hibernate uses these annotations to generate SQL and manage persistence automatically.

- **@Entity** : Tells JPA/Hibernate : "**This class represents a table in the database.**"
- **@Table** : Defines the **table name** and also can define **unique constraints**, **indexes**, **schema**.
- **@Id** : Marks a field as the **primary key**.
- **@GeneratedValue(strategy** = /*AUTO*/ **)** : Controls how the **primary key** is **generated**.
- **@CreationTimestamp** : Timestamp is generated when the **entity** is **first saved**.
- **@UpdationTimestamp** : Updates timestamp every time the **entity** is **updated**.

- **Table Annotation:**

```
@Table(
        name = "employees",
        catalog = "employee_catalog",
        schema = "hr",
        uniqueConstraints = {
                @UniqueConstraint(columnNames = {"email"})
        },
        indexes = {
                @Index(name = "idx_name", columnList = "name"),
                @Index(name = "idx_department", columnList = "department")
        }
)
```

- **name = "employees" :** Overrides the default table name (default = class name).
- **catalog = "employee_catalog" :** Defines the database **catalog**.
- **schema = "hr" :** Defines the database **schema**.
- **uniqueConstraints :** Defines table-level **unique constraints**.
- **indexes :** Creates **database indexes** for faster query performance.

- **Key features of JPA:**

1. **Entity Management:** Defines how **entities** (Java objects) are **persisted** to the database.
2. **Query Language:** Provides **JPQL** (Java Persistence Query Language) for querying entities.
3. **Transactions:** Manages **transactions**, making it easier to handle database operations within a transactional context.
4. **Entity Relationships:** Supports defining **relationships** between **entities** (e.g., One-to-One, One-to-Many, Many-to-One, Many-toMany).